Here we have two strings, both of which are valid UTF-8 (all ASCII remains valid when interpreted as UTF-8), but only one of which is also valid ASCII. With a detect order of ASCII > UTF-8, we can see which is which: The string containing the Unicode escape `"\u{141}"` will be encoded as *0xC5 0x81* (where `default_charset` is set to UTF-8). *0xC5* is outside the 7-bit range for ASCII but is a continuation byte in UTF-8. `mb_detect_encoding()` essentially counts "demerits" against encoding types when evaluating the input string: Invalid bytes and sequences score a demerit; the encoding with the fewest is the likeliest candidate (unless running in strict mode, as we'll see below). If we swap the order of encoding names, we get a different answer:

```
// could also be passed in as ['UTF-8', 'ASCII']
mb_detect_order('UTF-8,ASCII') . PHP_EOL;
print_r(mb_detect_order()) . PHP_EOL;
print mb_detect_encoding("Single bytes");
```

```
Output:
Array
(
    [0] => UTF-8
    [1] => ASCII
)
UTF-8
```

When evaluated as UTF-8, an ASCII string scores no "demerits," and because we asked for UTF-8 first, that's the answer returned by `mb_detect_encoding()`. Let's turn our attention to that function next.

Inspection

Checking that encoding is valid is quite important for some applications. There is the obvious problem of serving broken data back to clients; we don't want rogue question marks, squares, or random characters from different alphabets appearing in our application data. Another, rather more serious problem is one of security. Systems are normally tiered, or layered; a PHP application today usually relies on other systems for doing the heavy lifting with correctly directing network traffic. PHP prefers UTF-8 everywhere (it's even part of the PSR-1 standard), but there is a chance that other systems are using an alternative.

`mb_detect_encoding()`